

Architecture Document and Construction Plan

ExpenseMate – Personal Expense Manager

Software Construction, Assignment 1 | Phase 2: Architecture & Construction Planning | Version 1.0, 27 September 2026

1. Architectural Style

Style	Decision	Reason
Client–Server	Chosen (system level)	One server owns data and rules; validation and login cannot be bypassed by the client.
Layered	Chosen (inside the server)	Routes → controllers → services → repositories. Each layer is testable alone; a database change touches one layer.
Peer-to-Peer	Rejected	A personal app has no peers; syncing and securing peers adds cost with no benefit.

2. Logical View

Every record belongs to a user, and every query is filtered by that user (FR-04).

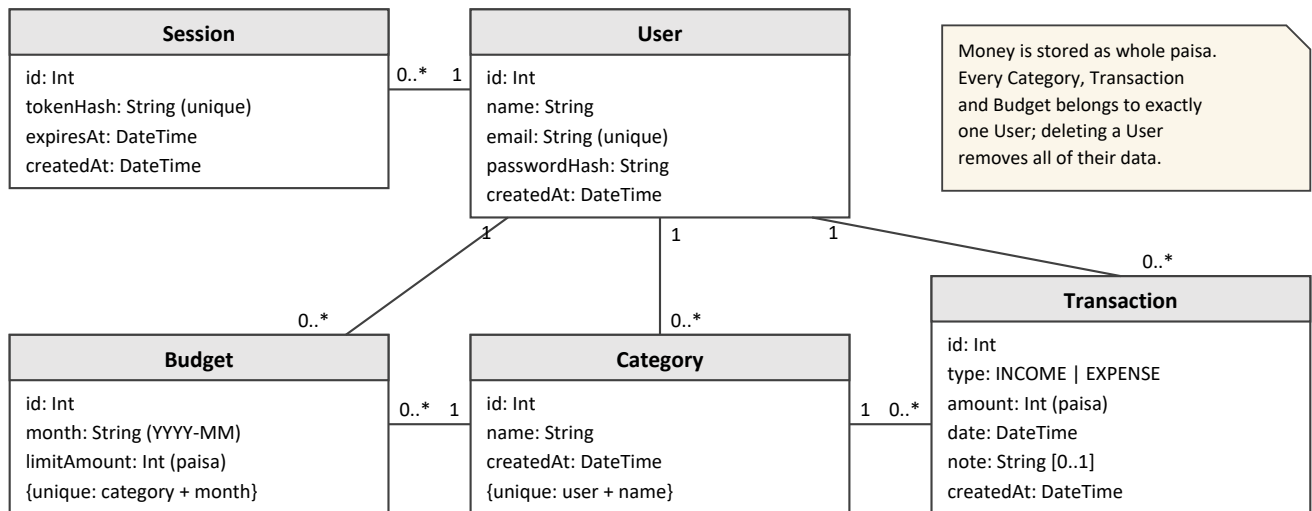


Figure 1. Domain model

Dependencies point downwards only; authentication, validation and error handling are shared by all layers.

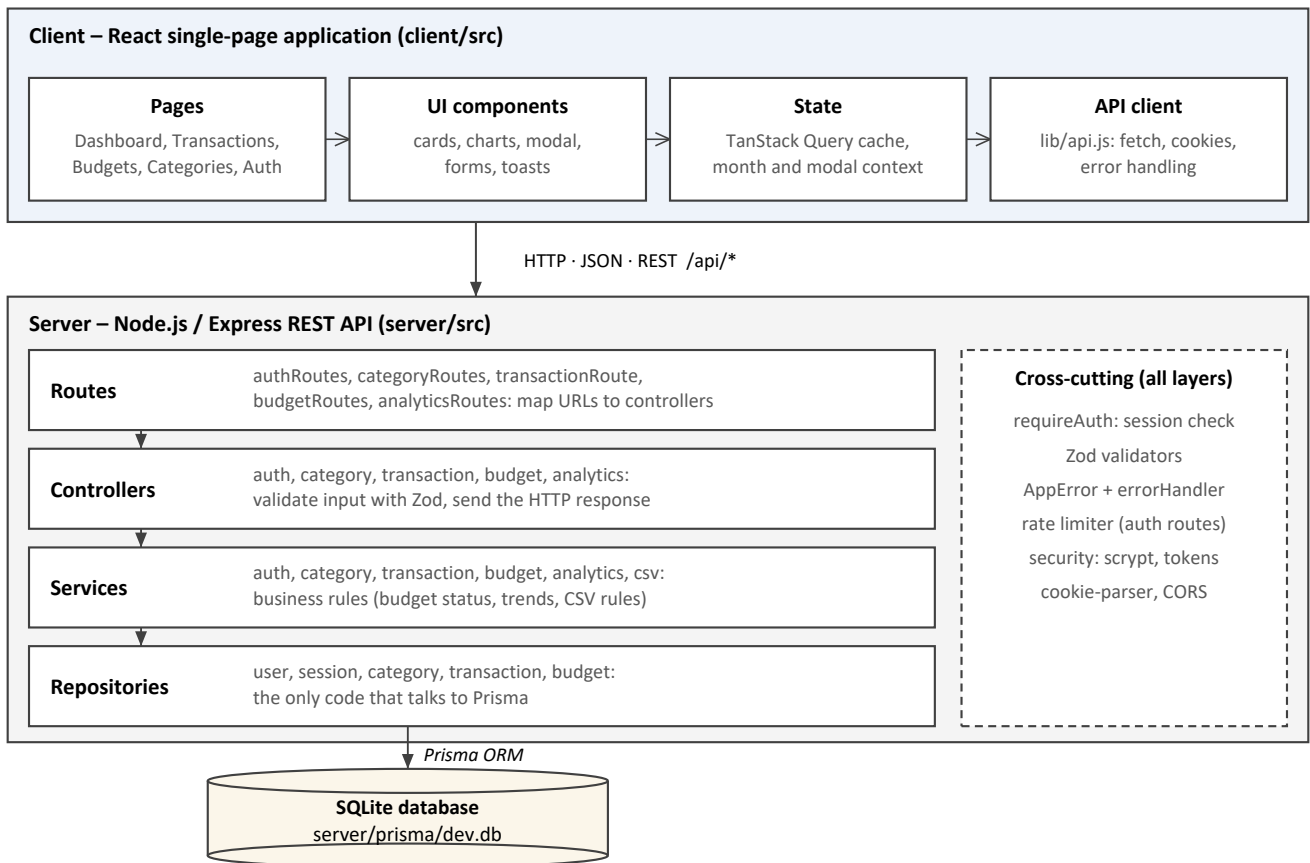


Figure 2. Client and server layers

3. Process View

A request to record an expense, from browser to database. Missing session → 401; invalid body → 400.

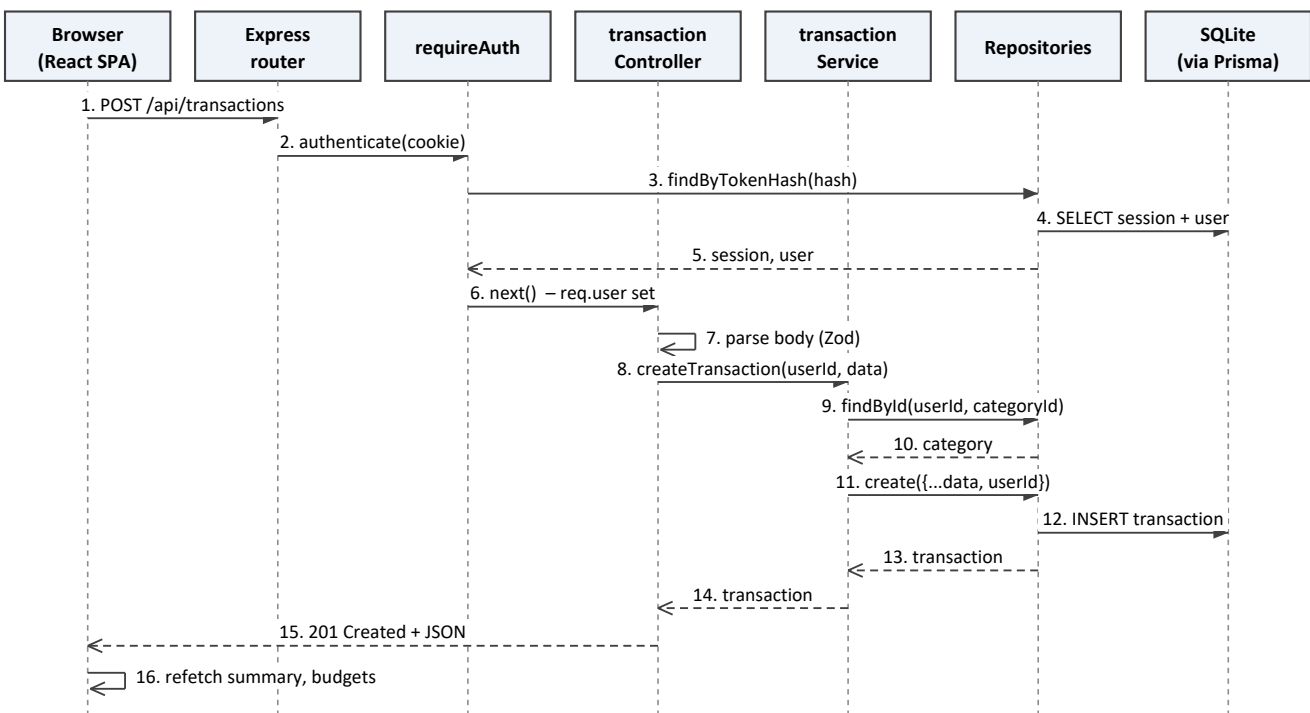


Figure 3. Sequence: record a transaction

- One Node.js process with asynchronous I/O; requests are stateless and the session is checked on each one.
- CSV import runs in one database transaction, so a failed import saves nothing.
- After each change the client refetches the affected data, so all screens stay consistent.

4. Physical View

Everything runs on one laptop. Vite forwards `/api` to the API, so the browser sees one origin. Code is shared through GitHub; databases are not.

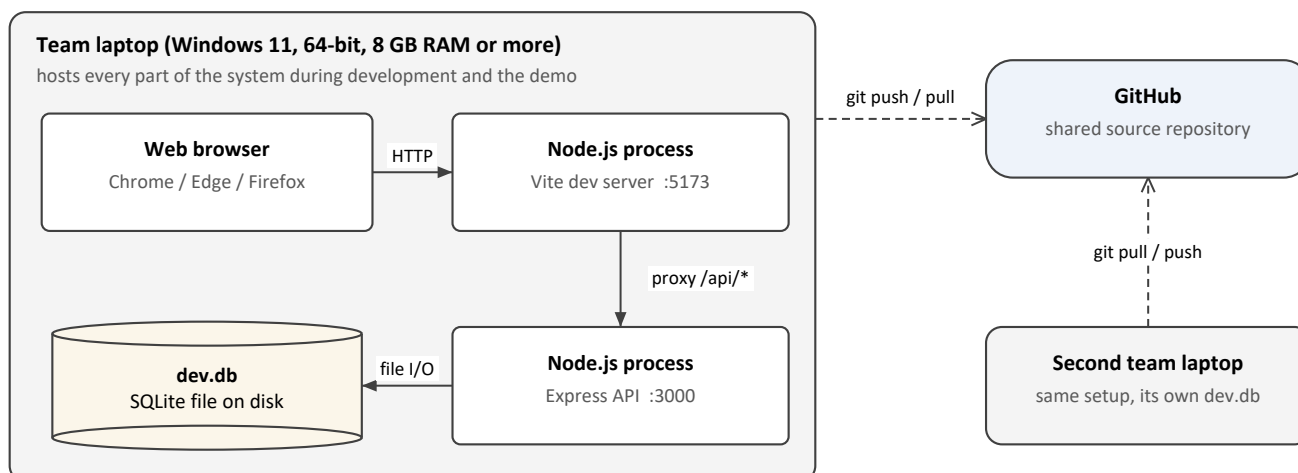


Figure 4. Machines, processes and connections

5. Deployment View

Client device and application host are the same laptop locally, but can be split without design changes.

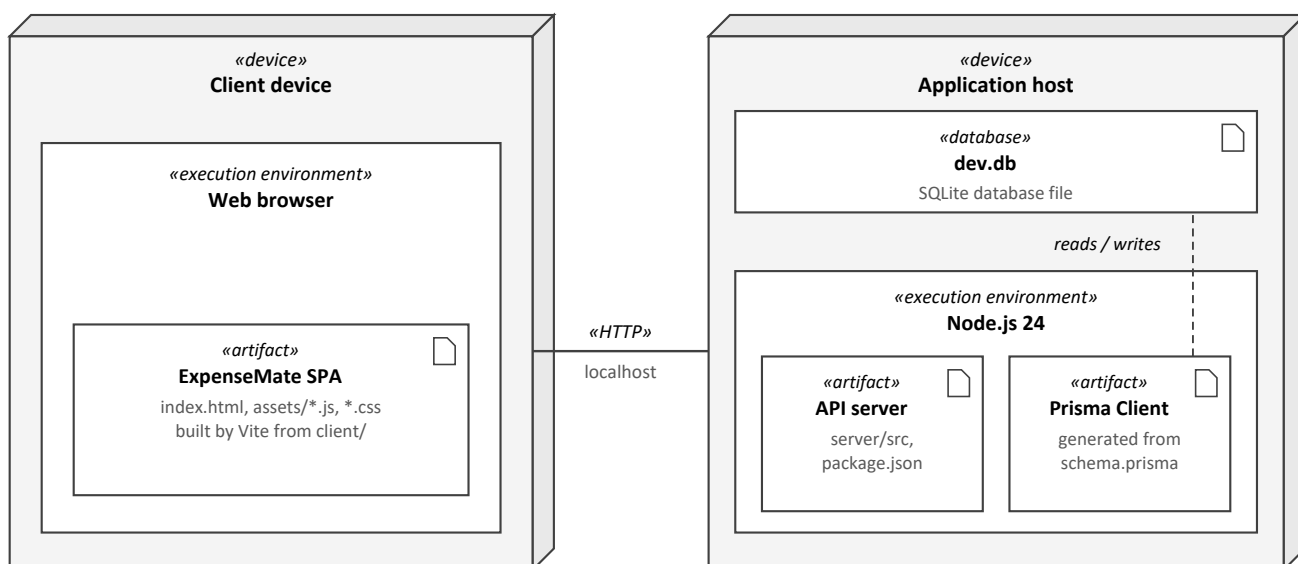


Figure 5. UML deployment diagram

Artifact	Source	Started with
Web app	client/src	npm run dev (or npm run build)
API server	server/src	npm run dev / npm start
Database	server/prisma	npx prisma migrate deploy; npm run db:reset for demo data

6. Construction Plan

6.1 Coding standards

Area	Rule
Style	4-space indent, double quotes, semicolons; CommonJS on the server, ES modules in the client.
Naming	camelCase functions, PascalCase components, UPPER_SNAKE_CASE constants; files named entity + layer (budgetService.js).
Layers	Routes map URLs, controllers handle HTTP, services hold rules, only repositories use Prisma.
Errors	Zod validates all input; services throw AppError; one middleware formats every error.
Data	Money in integer paisa; dates in UTC; months as YYYY-MM; queries filtered by user.

Area	Rule
Quality	async/await, small functions, clear names instead of comments; secrets only in .env.
Testing and Git	Unit tests with mocked repositories, integration tests on a test database; small commits; tests pass before each commit.

6.2 Tools

Purpose	Tool (version)
Runtime and server	Node.js 24.16, Express 5.2
Database	Prisma 6.19, SQLite
Validation and security	Zod 4.6, node:crypto (scrypt, SHA-256), cookie-parser 1.4, express-rate-limit 8.7
CSV	PapaParse 5.7
Client	React 19.3, React Router 7.18, Vite 8.3, TanStack Query 5.104, Recharts 3.10, Motion 13.4
Testing and metrics	Jest 30.5, Supertest 7.3, ESLint complexity rule
Collaboration	Git 2.54, GitHub, VS Code

6.3 Risks

Risk	Mitigation
Scope creep	Scope fixed by the SRS; optional features moved to Phase 5.
Client and API drift apart	One API contract, integration tests for every endpoint.
SQLite locked during migrations	Stop the API first; the seed script can rebuild the database.
Data leaks between users	Hashing, HttpOnly cookies, rate limiting, isolation tests.

6.4 Timeline

Five weeks; phases overlap where safe. Diamonds are milestones; weekends are shaded.

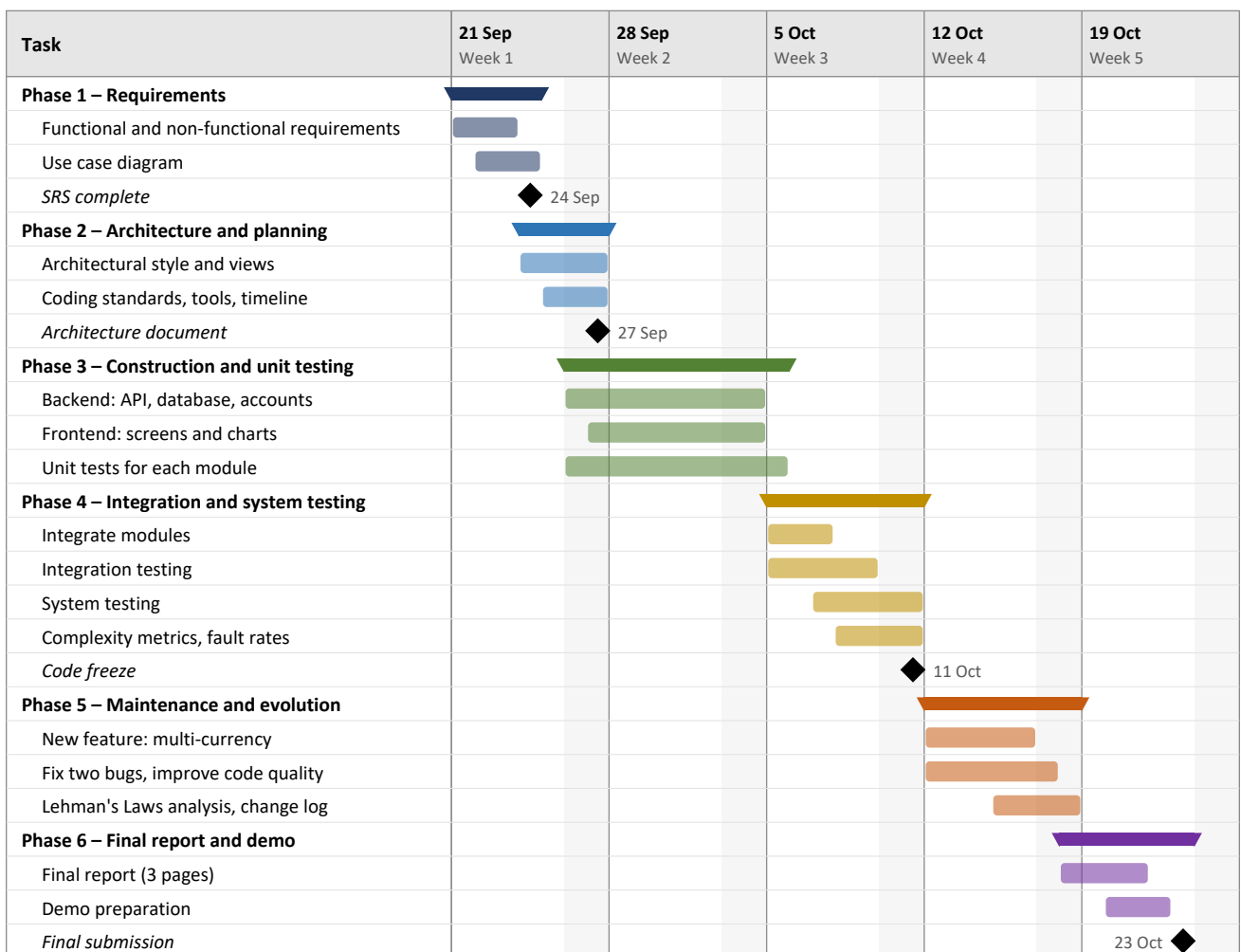


Figure 6. Gantt chart, 21 September – 23 October 2026